

Reproducible Writing with Quarto

A two-hour, hands-on workshop

Michael Lydeamore

2026-09-23

Table of contents

Welcome	5
One source, trustworthy results, many outputs	5
By the end	5
What you will make	5
Illustrative running order	6
How to use this book	6
Workshop details	6
1 Before the workshop	7
1.1 Install the tools	7
1.2 Prepare the project	7
1.3 Install the R packages	7
1.4 Run the preflight	8
1.5 Make the first render	8
1.6 Bring on the day	8
2 Reproducible writing	9
2.1 A familiar handoff	9
2.2 Source, process, and output	9
2.3 Why absolute paths fail	10
2.4 The project root	12
2.5 Separate inputs from products	12
2.6 Four layers of reproducibility	13
2.7 Clean sessions expose hidden dependencies	14
2.8 A practical handoff contract	14
2.9 Try the handoff yourself	15
3 Writing in Markdown	16
3.1 Source versus appearance	16
3.2 Document metadata	16
3.3 Headings create document structure	17
3.4 Section numbering and cross-references	18
3.5 Paragraphs and emphasis	19
3.6 Lists	19
3.7 Links, images, and footnotes	20
3.8 Quotations and block quotes	20
3.9 Callouts	21
3.10 Tables written in Markdown	21
3.11 Mathematics and numbered equations	22
3.12 Citations and bibliographies	22
3.13 A fuller Markdown exercise	23

4	Literate computation	24
4.1	Anatomy of an R cell	24
4.2	The execution model	25
4.3	Cell options control execution and presentation	25
4.4	Where options can be set	28
4.5	Inline computation	29
4.6	Figures, options, and cross-references	30
4.7	Tables and cross-references	31
4.8	Results produced “as is”	32
4.9	Errors during rendering	32
4.10	Randomness	33
4.11	Caching and freezing	33
4.12	Choosing options for the reader	33
4.13	A substantial computation exercise	34
5	Research publishing	35
5.1	Different kinds of research writing	35
5.2	What the outputs actually look like	35
5.3	One source with common and format-specific options	42
5.4	Format is not content	43
5.5	HTML, PDF, and Word serve different workflows	44
5.6	Numbering and cross-references survive reformatting	44
5.7	Citation processing	45
5.8	Author and affiliation metadata	45
5.9	What a journal extension contains	46
5.10	Starting a journal article versus adding a format	46
5.11	Comparing standard and journal output	47
5.12	Conditional content	47
5.13	Page and figure problems are diagnostic	47
5.14	Reproducible submission workflow	48
5.15	Publishing exercise	48
6	Bonus: prove it reruns	49
6.1	Clean-session check	49
6.2	Reproducibility checklist	49
6.3	Record the environment	50
7	Next steps	51
7.1	Apply the workflow to real work	51
7.2	Add safeguards when they solve a problem	51
7.3	Keep these habits	51
7.4	Resources	52
	Appendices	53
A	Markdown and cell-option reference	53
A.1	Markdown	53
A.2	Code cells	53

A.3	YAML	54
A.4	Terminal commands	54
B	Troubleshooting	55
B.1	Read the first useful error	55
B.2	<code>quarto: command not found</code>	55
B.3	An R package is missing	55
B.4	The CSV cannot be found	55
B.5	YAML parsing fails	56
B.6	PDF will not render	56
B.7	A figure reference shows ?	56
B.8	It worked interactively but not when rendered	56
B.9	Recovery route	57
C	Python and Jupyter translation	58
C.1	Preflight	58
C.2	Python cell	58
C.3	Inline Python	58
C.4	Figure	59
C.5	Concepts that transfer directly	59
D	Accessibility checklist	60
D.1	Structure	60
D.2	Figures	60
D.3	Colour	60
D.4	Tables and code	60
D.5	Slides	61
D.6	Final check	61
E	Beyond the workshop	62
E.1	Git	62
E.2	<code>renv</code>	62
E.3	Automated rendering	62
E.4	Quarto manuscripts	62
E.5	Parameters and profiles	63

Welcome

One source, trustworthy results, many outputs

In this workshop you will build a small research manuscript in which the prose, analysis, figures, tables, and references live together. You will render it to HTML and PDF.

By the end

You will be able to:

- explain literate programming in practical terms;
- write structured prose in Markdown;
- combine R code and narrative in a `.qmd` file;
- use relative paths, citations, figures, tables, and cross-references;
- render one source document to HTML and PDF
- start your PhD thesis using the [quarto template](#)

What you will make

The workshop project contains a short manuscript analysing a synthetic training study.

The data are synthetic, small, and stored inside the project. No network connection, account, or API key is required during the activities.

! Download before the workshop

- [Participant starter project](#)
- [Completed project](#)

Work from the starter project. Keep the completed project closed unless you need to recover from a missed step or want to compare your result at the end.

Illustrative running order

The detailed chapters are also designed to work as a reference after the workshop. They contain more examples than need to be demonstrated live, so the facilitator can select material and adjust the timings for the group.

Time	Topic	Product
00:00–00:15	Setup checkpoint	First successful render
00:15–00:30	Reproducible writing	Project map
00:30–00:50	Markdown	Structured manuscript
0:50–01:05	Break	
01:05–01:35	Computation	Live figure, table, and inline result
01:35–02:00	Research publishing	HTML, PDF, and journal-format demo

How to use this book

Keep the book open beside your editor. Green **Try it** boxes are the main hands-on path. Yellow **Checkpoint** boxes describe what should work before the room moves on. Purple **Discussion** boxes are brief pauses for interpretation. Blue **If you finish early** boxes are optional and can be skipped without affecting later exercises.

The main path assumes R. A compact [Python/Jupyter translation](#) is provided for later use.

Workshop details

- **Institution:** Monash University
- **Presenter:** Harriet Mason
- **Date:** today
- **Contact:** harriet.mason1@monash.edu

1 Before the workshop

Complete this page before the session. The workshop starts from a working installation rather than spending teaching time on downloads.

1.1 Install the tools

You need:

1. **Quarto 1.9 or later** from quarto.org.
2. **R 4.3 or later** from [CRAN](https://cran.r-project.org).
3. An editor with Quarto support: RStudio, Positron, or VS Code.
4. A LaTeX installation for PDF. TinyTeX is the smallest supported route for most participants.

If you do not already have TinyTeX, run this once in R:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

Restart your editor after installing Quarto or TinyTeX.

1.2 Prepare the project

1. Download the [starter project](#).
2. Extract it to a short, writable path.
3. Open the extracted folder as an RStudio/Positron/VS Code project or workspace.
4. Open a terminal **inside that folder**.

Do not work inside the zip file, a mail attachment preview, or a temporary downloads viewer.

1.3 Install the R packages

From the project terminal, run:

```
Rscript setup.R
```

This installs only packages that are missing: `knitr`, `rmarkdown`, `readr`, `dplyr`, and `ggplot2`.

1.4 Run the preflight

```
Rscript preflight.R
```

The check reports:

- Quarto and R versions;
- required R packages;
- a detected LaTeX engine; and
- whether a tiny HTML and PDF document can render.

1.4.1 Checkpoint before arrival

The final line reads **Preflight passed**. If HTML works but PDF fails, follow the [PDF troubleshooting steps](#) before the workshop.

1.5 Make the first render

```
quarto render manuscript.qmd --to html
```

Open `manuscript.html`. It will be incomplete—that is intentional—but it should contain a title, introduction, and activity placeholders.

Note

You can use your editor's **Render** button instead of the terminal. The workshop shows terminal commands because they behave consistently across editors.

1.6 Bring on the day

- the extracted starter project;
- the completed preflight result; and
- your laptop and power adapter

2 Reproducible writing

Reproducible writing is the practice of keeping claims, calculations, figures, tables, and their sources connected closely enough that the document can be rebuilt and checked. Quarto provides the machinery; good project habits make the machinery portable.

2.1 A familiar handoff

“It works on my computer”

A colleague sends you a folder called `final_analysis_revised_3`. Their email says:

Open `analysis.R`, change the first `setwd()` line to wherever you saved the folder, run lines 1–287, ignore the warnings, then open `paper.docx`. Figure 2 should match the plot in the `figures-old` folder, except the number in the Results section was updated manually.

Before checking the research, you must reconstruct the colleague’s computer: their folders, execution order, interactive objects, package versions, and copy-and-paste history. Even if every statistical command is correct, the handoff is fragile.

Most researchers have either received a project like this or created one under deadline pressure. Reproducible writing tries to make the handoff instead:

1. Open the project folder.
2. Install the documented requirements.
3. Render `manuscript.qmd`.
4. Inspect the source, output, and any reported differences.

That is a much smaller and testable process.

2.2 Source, process, and output

Literate programming places human explanation and executable instructions in one source document (Knuth 1984). Quarto coordinates the rendering process:

```
source data + .qmd + software environment
      |
      render
      |
HTML / PDF / Word / slides / tutorials
```

The rendered PDF is not the reproducible object by itself. It is evidence that a particular source was rendered in a particular environment. The useful object to share is the project that can recreate it.

This changes the role of common files:

File	Role	Usually edited by
data/training-study.csv	Input	Data producer or documented cleaning step
manuscript.qmd	Narrative and analysis instructions	Author
bibliography.bib	Citation metadata	Author/reference manager
manuscript.html	Rendered product	Quarto
manuscript.pdf	Rendered product	Quarto and LaTeX

If a rendered product looks wrong, repair the source or process and render again. **Editing the product directly creates an undocumented fork.**

2.3 Why absolute paths fail

An absolute path starts from a computer-specific root:

```
# macOS
readr::read_csv("/Users/alex/Desktop/project/data/results.csv")

# Windows
readr::read_csv("C:/Users/alex/OneDrive - University/project/data/results.csv")

# Linux or a research server
readr::read_csv("/home/alex/projects/project/data/results.csv")
```

The path may be correct today, but it encodes facts that are not properties of the research:

- the operating system;
- the author's username;
- the name of a university or cloud-storage service;
- where the author happened to save the project;
- which drive or network mount was available; and
- sometimes confidential project, client, or participant names.

When someone else receives the project, at least one of those facts changes. The code fails before it reaches the analysis.

2.3.1 The same author can still break it

Absolute paths are not only a collaboration problem. They fail when you:

- move the project from Downloads to Documents;
- rename a parent folder;
- switch from a desktop to a laptop;
- restore from a backup;
- run on a high-performance computer;
- change institutions or cloud-storage accounts; or
- ask an automated build service to render the document.

An absolute path is therefore a claim that *this project will always live in this exact location on this exact kind of machine*.

2.3.2 `setwd()` moves the problem

A common repair is:

```
setwd("/Users/alex/Desktop/project")
```

The remaining paths may now look relative, but every collaborator still has to edit the script before it can run. Worse, `setwd()` changes global session state: later code depends on whether that line ran and what the working directory was beforehand.

Opening a project establishes one shared root without hard-coding each user's computer.

2.3.3 Relative paths are the solution

Inside the workshop project, write:

```
readr::read_csv("data/training-study.csv")
```

This path means:

1. start from the project root;
2. enter the `data` folder; and
3. open `training-study.csv`.

It describes the relationship between files, which remains true when the whole project is moved.

Path	Meaning
<code>data/file.csv</code>	A file below the project root
<code>figures/result.png</code>	A generated or supplied figure folder
<code>scripts/clean.R</code>	A script below the root

Path	Meaning
<code>../shared/file.csv</code>	A file outside the project — portable only if that surrounding structure is also shared
<code>/data/file.csv</code>	An absolute path from the filesystem root, not the project

 Relative does not mean “any path without a username”

`../someone-elses-folder/data.csv` is syntactically relative but still depends on a directory outside the shared project. Prefer keeping shareable inputs inside the project, or document and validate the external data location explicitly.

2.4 The project root

Your editor and Quarto need a stable answer to “relative to what?” For this workshop, the extracted folder containing `_quarto.yml` is the project root:

```
quarto-workshop-starter/
  _quarto.yml
  manuscript.qmd
  bibliography.bib
  data/
    training-study.csv
    README.md
  _extensions/
```

Open the **folder**, not just an isolated `.qmd` file. Run terminal commands from that folder. Quarto then has a consistent base for documents and resources.

You can see R’s current working directory with:

```
getwd()
```

Use this for diagnosis, not as a value to copy into `setwd()`.

2.5 Separate inputs from products

A project becomes easier to understand when its folders communicate intent:

```
project/  
  data/          # original or documented inputs  
  R/             # reusable functions, if the project grows  
  figures/       # supplied figures; generated figures may be temporary  
  manuscript.qmd # source  
  bibliography.bib  
  README.md      # how to reproduce the project
```

Useful rules are:

- do not overwrite raw inputs during analysis;
- derive cleaned variables in code;
- give files stable, descriptive names;
- keep secrets and confidential data out of distributable projects;
- avoid folders named only `old`, `new`, `final`, or `misc`; and
- make it obvious which files are sources and which are generated.

2.6 Four layers of reproducibility

Keeping data and code together is necessary but not sufficient.

2.6.1 1. Computational

Can the recorded instructions recreate the values, figures, and tables?

Common failures include missing code, interactive objects, manual spreadsheet edits, and undocumented execution order.

2.6.2 2. Environmental

Can the instructions run with the available versions of R, packages, Quarto, LaTeX, and system libraries?

The workshop records `sessionInfo()`. Package lockfiles such as `renv` provide a stronger environment specification.

2.6.3 3. Provenance

Can a reader identify where data and external information came from, what was changed, and under what permissions?

A mysterious `final.csv` is not made trustworthy merely by reading it with reproducible code.

2.6.4 4. Narrative

Do the prose, captions, and conclusions accurately describe the computations?

Code can keep `n = 40` synchronised with a dataset. It cannot decide whether a causal claim is justified. Reproducibility makes reasoning inspectable; it does not replace judgement.

2.7 Clean sessions expose hidden dependencies

Suppose you created `study_clean` interactively yesterday, then wrote:

```
mean(study_clean$change)
```

The line works while that object remains in your Global Environment. It fails on a colleague's computer because the document never creates `study_clean`.

A clean render starts a new execution context and runs the document in order. This exposes:

- missing package calls;
- objects created only interactively;
- files read from the wrong location;
- dependence on a particular execution order; and
- values copied rather than calculated.

The failure is useful evidence about what the source has not yet documented.

2.8 A practical handoff contract

Before sharing a project, ask whether a colleague can answer:

1. Which file should I open first?
2. Where are the inputs, and may they be shared?
3. Which command recreates the outputs?
4. Which software and packages are required?
5. Which files should never be edited manually?
6. What result should I see if the render succeeds?

The workshop's `README.md`, `setup.R`, `preflight.R`, and `.qmd` sources provide those answers at a modest level. More complex projects may add a package lockfile, tests, workflow manager, or container.

2.9 Try the handoff yourself

Open `manuscript.qmd` and `data/training-study.csv`.

1. Find the code that reads the CSV.
2. Confirm that the path contains no username or drive letter.
3. Move a **copy of the entire project folder** to a different location.
4. Open the copied folder as a project and render the manuscript.
5. Move only `manuscript.qmd` without its project and observe what is missing.
6. Explain why the first move succeeds and the second is not a complete handoff.

2.9.1 Checkpoint

You should be able to describe the project as a self-contained set of relationships between files—not as a folder that happens to work from one person’s Desktop.

In Chapter 3 we turn the narrative itself into structured, portable source. In Chapter 4 we make calculated values and outputs part of the same rendering process.

3 Writing in Markdown

Markdown is plain text with punctuation that describes **structure** rather than appearance. You identify something as a heading, quotation, citation, or code expression; the output format decides how that structure should look.

That separation is what allows the same source to become a web page, PDF, Word document, book chapter, slide deck, or tutorial.

3.1 Source versus appearance

In a word processor, “make this 18-point bold” is a visual instruction. In Markdown, **## Methods** says “this is a second-level heading.” Quarto can then:

- number it in a PDF;
- put it in an HTML table of contents;
- turn it into a slide boundary;
- include it in navigation; and
- give it an address for cross-referencing.

Semantic structure is both more reusable and more accessible than manual visual formatting.

3.2 Document metadata

A Quarto document begins with YAML metadata between `---` lines:

```
---
title: "A reproducible analysis"
subtitle: "A synthetic training study"
author: "A. Researcher"
date: today
bibliography: bibliography.bib
format:
  html:
    toc: true
  pdf:
    number-sections: true
---
```

The YAML is configuration. It answers questions such as:

- What is the document called?
- Who wrote it?
- Which bibliography should be used?
- Which outputs should be created?
- Which options differ between those outputs?

Indentation defines nesting:

```
format:
  html:          # belongs under format
    toc: true    # belongs under html
  pdf:          # also belongs under format
  number-sections: true
```

If `toc` were aligned with `format`, it would have a different scope.

3.3 Headings create document structure

Use one `#` for a level-one heading, two for level two, and so on:

Source

```
# Introduction

## Background

## Research question

# Methods

## Data

## Analysis
```

Rendered structure

```
1 Introduction
  1.1 Background
  1.2 Research question
2 Methods
  2.1 Data
  2.2 Analysis
```

The precise numbering and typography come from the output format.

Do not skip from `#` to `###` simply because you prefer the smaller font. That creates a broken outline for navigation and assistive technology. Change the *theme* if the visual design needs adjustment (more on this later).

3.4 Section numbering and cross-references

Turn on numbering for an output:

```
format:
  html:
    number-sections: true
  pdf:
    number-sections: true
```

Give a heading a stable identifier:

```
## Statistical analysis {#sec-statistical-analysis}
```

Refer to it with:

```
The model specification is described in @sec-statistical-analysis.
```

Quarto writes “Section 3.2” or the appropriate number. If the section moves, the reference updates automatically.

Source

```
## Data preparation {#sec-data-preparation}

Missing values were retained for descriptive checks.

## Statistical analysis

See @sec-data-preparation before fitting the model.
```

Rendered idea

3.1 Data preparation

Missing values were retained for descriptive checks.

3.2 Statistical analysis

See **Section 3.1** before fitting the model.

Stable identifiers are better than writing “see the section above” or manually typing “Section 3.1”. Use short, descriptive, lowercase labels separated by hyphens.

Quarto uses the same system for sections, figures, tables, equations, listings, and exercises:

Object	Label prefix	Reference
Section	sec-	@sec-methods
Figure	fig-	@fig-change

Object	Label prefix	Reference
Table	tbl-	@tbl-summary
Equation	eq-	@eq-change
Listing	lst-	@lst-model

3.5 Paragraphs and emphasis

A blank line starts a new paragraph. A single line break in the source normally continues the same paragraph, allowing you to wrap source text without changing the rendered prose.

Source

```
This is one paragraph even though the
source wraps across two lines.
```

```
This is a new paragraph with *emphasis*,
**strong emphasis**, and `inline code`.
```

Rendered

This is one paragraph even though the source wraps across two lines.

This is a new paragraph with *emphasis*, **strong emphasis**, and `inline code`.

Use emphasis to convey meaning, not to simulate headings. Backticks are useful for filenames, functions, variables, and short commands.

3.6 Lists

Unordered lists use -; ordered lists use 1.. Markdown can number ordered lists automatically:

Source

```
The analysis will:
```

- read the source data;
- derive the change score; and
- produce a figure.

```
The reader should:
```

1. open the project;
1. render the manuscript;
1. inspect the output.

Rendered

The analysis will:

- read the source data;
- derive the change score; and
- produce a figure.

The reader should:

1. open the project;
2. render the manuscript;
3. inspect the output.

Using 1. for every ordered item makes rearranging source easier. Use explicit numbers only when the source number itself matters.

3.7 Links, images, and footnotes

```
[Quarto documentation](https://quarto.org/docs/guide/)

! [A concise description of the image](images/result.png)

This claim needs a qualification. [^qualification]

[^qualification]: The data are synthetic and illustrate workflow only.
```

Good link text tells the reader where the link goes. Avoid “click here”.

Image text inside `![...]` becomes alternative text. Describe the information a reader needs, not merely the filename or “image of a graph”.

Footnotes are appropriate for supplementary explanation. If information is central to interpreting the result, keep it in the main text.

3.8 Quotations and block quotes

Prefix quoted paragraphs with `>`:

Source

```
> Reproducibility is a property of a
> workflow, not simply a file format.
```

Rendered

Reproducibility is a property of a workflow, not simply a file format.

For published words, include a citation and page or section locator where appropriate. Markdown syntax does not remove scholarly attribution requirements.

3.9 Callouts

Callouts separate warnings, definitions, tips, or assumptions from the main narrative:

```
::: {.callout-warning}
The data are synthetic and must not be interpreted as evidence.
:::
```

Warning

The data are synthetic and must not be interpreted as evidence.

Common types are **note**, **tip**, **important**, **warning**, and **caution**. A callout should add meaning, not turn every paragraph into a coloured box.

3.10 Tables written in Markdown

Small, static tables can be written directly:

Source

```
| Format | Best suited to |
|---|---|
| HTML | Browsing and interaction |
| PDF | Fixed-page distribution |
| DOCX | Editorial workflows |
```

Rendered

Format	Best suited to
HTML	Browsing and interaction
PDF	Fixed-page distribution
DOCX	Editorial workflows

Use a code-generated table when values come from data. A hand-written Markdown table is appropriate for stable explanatory content such as option summaries.

3.11 Mathematics and numbered equations

Inline mathematics uses one pair of dollar signs:

```
The individual change is  $\Delta_i = y_{i1} - y_{i0}$ .
```

The rendered result is: the individual change is $\Delta_i = y_{i1} - y_{i0}$.

A display equation uses $\$$. Add a label inside braces for numbering and cross-referencing:

```
 $\$$   
 $\backslash\mathrm{bar}\{\Delta\}_g =$   
 $\frac{1}{n_g}\sum_{i=1}^{n_g}\Delta_i$   
 $\$$  {#eq-group-change}  
  
Group means are defined in @eq-group-change.
```

$$\bar{\Delta}_g = \frac{1}{n_g} \sum_{i=1}^{n_g} \Delta_i \quad (3.1)$$

Group means are defined in Equation [3.1](#).

3.12 Citations and bibliographies

Bibliographic records live in `bibliography.bib`. A record has a citation key:

```
@article{knuth1984,  
  author = {Knuth, Donald E.},  
  title = {Literate Programming},  
  year = {1984}  
}
```

Use the key in prose:

Source

```
Literate programming was formalised by  
@knuth1984.
```

```
Reproducibility requires more than  
retaining code [sandve2013].
```

Rendered

Literate programming was formalised by Knuth (1984).

Reproducibility requires more than retaining code (Sandve et al. 2013).

Quarto formats the citation and builds the reference list. A Citation Style Language (.cs1) file can change the style without rewriting citations.

Useful citation patterns include:

```
[@knuth1984]           # parenthetical
@knuth1984             # textual author
[@knuth1984, pp. 100-102] # locator
[@knuth1984; @sandve2013] # multiple sources
```

The bibliography automates formatting, not source evaluation. Check names, titles, dates, identifiers, and whether the cited source supports the claim.

Probably the easiest way to get bib keys is from [Zotero](#) and the [Better BibTeX extension](#) which can export the required format for a .bib file automatically.

3.13 A fuller Markdown exercise

In manuscript.qmd:

1. Complete the title, subtitle, author, and output options in YAML.
2. Create Introduction, Methods, Results, Discussion, and References sections.
3. Add stable `sec-` identifiers to Methods and Results.
4. Refer to Methods from the Results prose.
5. Add a research question, bulleted aims, and the numbered change-score equation.
6. Add one citation, one footnote, one link, and one warning callout.
7. Render to HTML and inspect the table of contents, numbering, and references.
8. Render to PDF and identify what changed visually and what remained semantically the same.

3.13.1 Checkpoint

The source contains structure rather than manual styling. Headings appear in navigation, section and equation references update automatically, and citations are generated from the bibliography.

Chapter 4 applies the same source-to-output principle to calculated values, figures, and tables.

4 Literate computation

An executable document contains three interleaved kinds of material:

1. **narrative**—the question, methods, interpretation, and limitations;
2. **code**—instructions for reading, transforming, analysing, and presenting data; and
3. **computed output**—values, messages, tables, and figures created when the document renders.

The rendered document is a record of code executed in context, not a screenshot of an interactive session.

4.1 Anatomy of an R cell

An executable R cell begins with ````${r}```` and ends with three backticks. Cell options sit at the top and begin with `#|`:

```
#| label: group-counts
#| echo: true
#| message: false

study |>
  count(group)
```

When this book renders, that pattern produces a live demonstration:

```
study |>
  count(group)
```

```
# A tibble: 2 x 2
  group      n
  <chr>    <int>
1 comparison    20
2 intervention    20
```

The cell has:

- an **engine** (`r`);
- a **label** (`demo-group-counts`);
- zero or more **options**;
- R code; and

- output generated during rendering.

Labels should be unique, descriptive, and stable. Labels with reserved prefixes such as `fig-` and `tbl-` create cross-referenceable outputs.

4.2 The execution model

For an R-based `.qmd`, Quarto asks `knitr` to execute cells from top to bottom. Objects created in an earlier cell are available later in that render.

```
# Cell 1
study <- readr::read_csv("data/training-study.csv")

# Cell 2: depends on Cell 1
nrow(study)
```

This has several consequences:

- changing cell order can change whether code works;
- a document should create every object it uses;
- packages should be loaded or namespaced in the source;
- interactive Global Environment objects are not reliable dependencies; and
- restarting R before rendering is a useful test.

4.2.1 A common hidden-state failure

Imagine running this interactively:

```
study_with_change <- study |>
  mutate(change = followup_score - baseline_score)
```

You then write a document cell containing only:

```
mean(study_with_change$change)
```

It works until R is restarted. The document contains the result-producing line but not the instruction that creates its input. A clean render reveals the missing step.

4.3 Cell options control execution and presentation

Chunk options are not cosmetic trivia. They determine what runs, what readers see, what can be referenced, and how output is presented.

4.3.1 Core execution and visibility options

Option	Typical value	Code runs?	Code shown?	Output shown?
eval	false	No	Yes	No
echo	false	Yes	No	Yes
output	false	Yes	Yes	No
include	false	Yes	No	No
warning	false	Yes	As configured	Warnings hidden
message	false	Yes	As configured	Messages hidden
error	true	Yes	As configured	Error displayed; render continues

These options answer different questions:

- Should the code run? `eval`
- Should readers see the code? `echo`
- Should readers see its normal output? `output`
- Should anything from the cell appear? `include`
- How should diagnostics be handled? `warning`, `message`, `error`

4.3.2 Default: code and output

Use the default when the calculation itself is part of the explanation:

```
study |>
  summarise(
    participants = n(),
    mean_change = mean(change)
  )
```

```
# A tibble: 1 x 2
  participants mean_change
      <int>         <dbl>
1         40          6.95
```

The reader can inspect the instructions and the result.

4.3.3 `echo: false`: show the result, hide the implementation

This is common in a polished reader-facing report:

```
# A tibble: 2 x 2
  group      n
  <chr>    <int>
1 comparison 20
2 intervention 20
```

The code still runs and remains in the `.qmd` source, but the rendered reader sees only the result. Hiding code is a presentation choice, not reproducibility by itself; the source must still be available to someone expected to audit it.

4.3.4 `eval: false`: show an example without running it

```
# An illustrative call that should not run during this render
download.file("https://example.org/large-data.csv", "data/large-data.csv")
```

This is useful for teaching installation commands, destructive examples, slow jobs, or code requiring credentials. Do not use it for code that supposedly created the reported result.

4.3.5 `output: false`: run code, hide normal output

```
summary_table <- study |>
  group_by(group) |>
  summarise(mean_change = mean(change), .groups = "drop")

summary_table
```

The object is created for later cells, and the code is visible, but the printed table is suppressed here.

4.3.6 `include: false`: setup that remains out of the document

The first cell in this chapter loaded packages and data using:

```
#| include: false

library(readr)
library(dplyr)
library(ggplot2)
study <- read_csv("materials/complete/data/training-study.csv")
```

The code ran, but neither code nor output appeared. This is appropriate for routine setup. It is not a place to conceal analytically important decisions.

4.3.7 Warnings and messages

Package startup messages rarely help a manuscript reader:

```
#| message: false  
#| warning: false
```

Suppress expected noise only after understanding it. A convergence warning, failed transformation, or removed observation may be evidence of an analysis problem. Fix or explain substantive warnings rather than hiding them globally.

4.3.8 Folding rather than removing code

HTML can keep code available without placing it in the main reading flow:

```
format:  
  html:  
    code-fold: true  
    code-summary: "Show the R code"
```

This affects HTML interaction. PDF cannot fold code, so decide separately whether code should be shown, hidden, or moved to an appendix.

4.4 Where options can be set

Options can apply at three levels.

4.4.1 Project level

In `_quarto.yml`:

```
execute:  
  warning: false  
  message: false
```

Use project defaults for consistent policy across documents.

4.4.2 Document level

In a document's YAML:

```
---  
execute:  
  echo: true  
  warning: false  
---
```

Use document defaults for the intended audience of that document.

4.4.3 Cell level

```
#| echo: false  
#| fig-width: 7
```

Cell options override broader defaults for one case. The practical precedence is:

```
project default < document default < cell option
```

Keep exceptions local and intentional. If every cell repeats the same option, move it to document or project YAML.

4.5 Inline computation

Inline code places a calculated value inside a sentence. The source pattern is:

The study contains ‘r nrow(study)’ participants.

In this rendered chapter, the live sentence is:

The study contains 40 participants.

Other source examples include:

- The mean change was ‘r round(mean(study\$change), 1)’ points.
- There were ‘r dplyr::n_distinct(study\$group)’ study groups.

Use short, readable expressions inline. If a calculation is complicated, create a clearly named object in a cell and insert only that object:

```
overall_mean_change <- mean(study$change)
```

The prose can then use:

The mean change was ‘r round(overall_mean_change, 1)’ points.

This keeps prose readable and makes the calculation easier to test.

4.6 Figures, options, and cross-references

A figure cell combines execution options with publication metadata:

```
#| label: fig-change-demo
#| fig-cap: "Change score by training hours and study group."
#| fig-alt: "Scatterplot of training hours against change score..."
#| fig-width: 7
#| fig-height: 4.5
#| dpi: 300

ggplot(study, aes(training_hours, change, colour = group, shape = group)) +
  geom_point() +
  geom_smooth(method = "lm", se = FALSE)
```

The live output is [Figure 4.1](#):

```
ggplot(study, aes(training_hours, change, colour = group, shape = group)) +
  geom_point(size = 2.5) +
  geom_smooth(method = "lm", se = FALSE) +
  labs(
    x = "Training hours",
    y = "Change in score",
    colour = "Study group",
    shape = "Study group"
  ) +
  theme_minimal()
```

``geom_smooth()`` using formula = 'y ~ x'

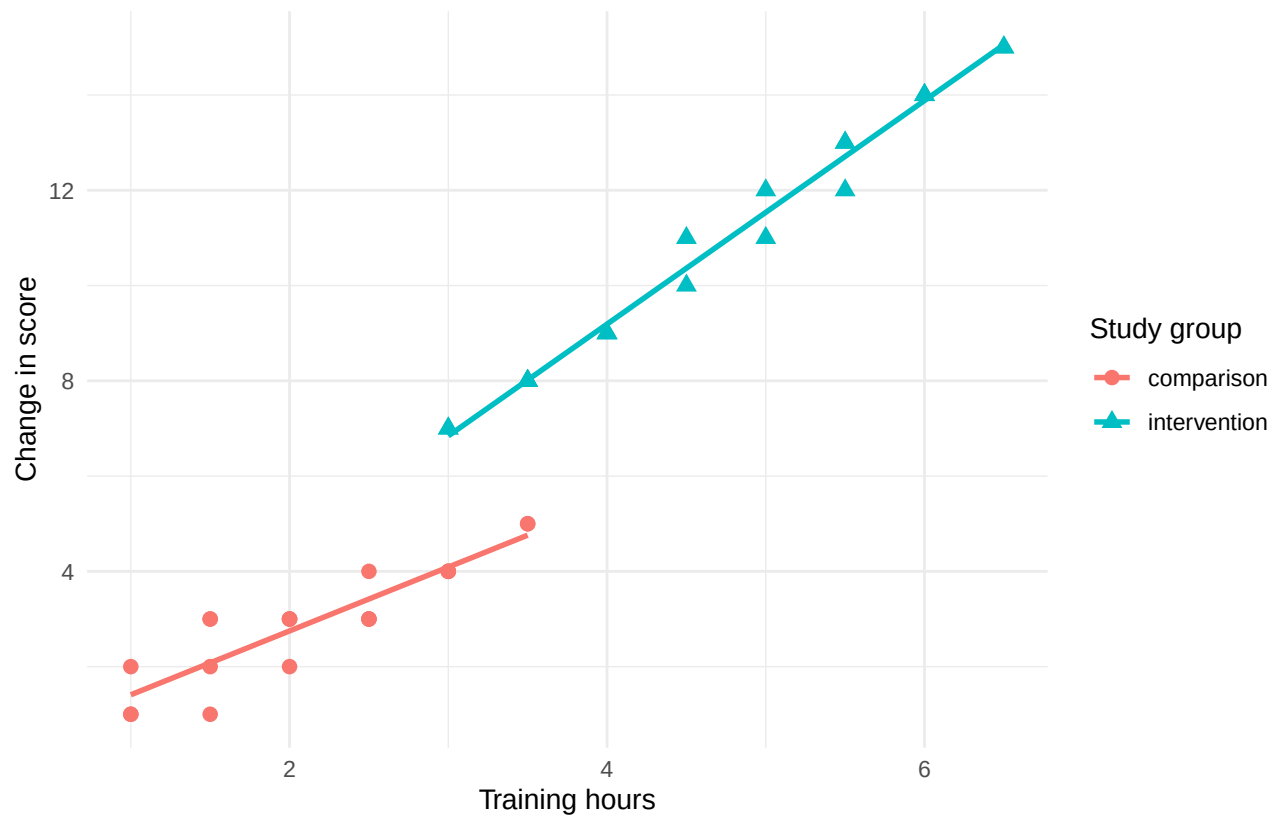


Figure 4.1: Change score by training hours and study group.

Important figure options include:

Option	Purpose
<code>label: fig-name</code>	Creates a stable cross-reference target
<code>fig-cap</code>	Visible caption
<code>fig-alt</code>	Accessible description of information in the figure
<code>fig-width, fig-height</code>	Size of the plotting device in inches
<code>dpi</code>	Raster resolution
<code>fig-align</code>	Alignment in the output
<code>fig-show: hold</code>	Holds multiple plots from one cell together

Plot dimensions affect the space in which the plot is drawn, not merely how a finished image is stretched. Text, facets, and legends may reflow when dimensions change.

4.7 Tables and cross-references

Create the data object in code:

Then present it with a caption and table label:

```
knitr::kable(group_summary, digits = 1)
```

Table 4.1: Change scores by study group.

group	participants	mean_change	sd_change
comparison	20	3.0	1.2
intervention	20	10.9	2.7

Table 4.1 is numbered and referenced from prose. If another table is inserted earlier, Quarto updates the number.

Keep a distinction between:

- the **data object** (`group_summary`);
- the **presentation function** (`knitr::kable()`); and
- the **interpretation** in prose.

This makes it easier to change table styling without recalculating the result.

4.8 Results produced “as is”

Some R functions deliberately return Markdown or HTML. With knitr, `results: 'asis'` or Quarto's `output: asis` can ask the renderer to interpret returned markup rather than print it literally.

Use this only when the code is intentionally generating document structure. Untrusted text must not be inserted as raw HTML, and format-specific markup may fail in PDF or Word.

4.9 Errors during rendering

By default, an error stops the render. This is usually correct: a document with a failed analysis should not silently produce a polished output.

For a teaching document that intentionally demonstrates an error:

```
#| error: true  
  
mean("not numeric")
```

The error appears in the rendered lesson and the document continues. Do not set `error: true` globally in a production manuscript merely to obtain an output.

4.10 Randomness

Simulations, sampling, model initialisation, and some algorithms use random numbers. Without a seed, repeated renders may differ:

```
set.seed(20260729)
sample(1:100, 5)
```

A seed makes the pseudorandom sequence repeatable. It does not make a simulation design appropriate or guarantee identical results across every software implementation.

4.11 Caching and freezing

Two related features can avoid unnecessary computation:

- **cell caching** can reuse a cell's previous computational result; and
- **Quarto freeze** can reuse rendered computational Markdown for a document.

Examples:

```
execute:
  cache: true
```

```
execute:
  freeze: auto
```

These improve rendering speed, but cached output can conceal missing external dependencies during testing. Before release, perform at least one clean, uncached render in a fresh copy or controlled environment.

Never use caching as a substitute for recording inputs.

4.12 Choosing options for the reader

The same analysis can be presented differently:

Audience	Reasonable presentation
Analyst collaborator	Show code and output
Journal reader	Hide routine setup; show methods-critical code selectively
Auditor or reviewer	Provide source or folded code plus environment details
Student	Show starter code; reserve answers for solution output

Audience	Reasonable presentation
Lecture audience	Hide most code and enlarge the explanatory figure

Chunk options let one computational workflow support these audiences, but a document should not pretend that hidden source does not exist.

4.13 A substantial computation exercise

In `manuscript.qmd`:

1. Add an `include: false` setup cell that loads packages and reads the CSV.
2. Derive `change` in code.
3. Replace typed sample-size and group-count values with inline R.
4. Create a summary object in a hidden setup cell.
5. Present it in a `tbl-` cell with a caption and refer to it from prose.
6. Create a `fig-` cell with a caption, informative alt text, explicit dimensions, and both colour and shape mappings.
7. Use `code-fold` for HTML and decide explicitly how code appears in PDF.
8. Restart R and render without running any cells manually.
9. Change one follow-up value in a temporary copy of the CSV and list every output that updates.
10. Restore the source data and render again.

4.13.1 Checkpoint

You can explain the difference between `eval`, `echo`, `output`, and `include`; identify which cell creates every object; and trace a reported value back to the source data and code that generated it.

5 Research publishing

Quarto is not one document style. It is a publishing system that can apply different output conventions to the same structured source.

5.1 Different kinds of research writing

The underlying Markdown and computation skills transfer across several products:

Product	Typical reader	Important features
Analysis note	Yourself or close collaborators	Visible code, diagnostics, rapid iteration
Research report	Client, team, or decision maker	Navigation, executive summary, branded output
Journal manuscript	Editors, reviewers, scholarly readers	Author metadata, citations, numbered objects, submission template
Reproducible manuscript site	Scholarly readers and auditors	Article plus linked computational notebooks and resources
Book or technical manual	Learners and long-form readers	Chapters, navigation, cross-chapter references, search
Teaching practical	Students and instructors	Exercises, starter code, controlled solutions
Presentation	Live audience	Large text, staged explanation, speaker notes

These are not merely different file extensions. They have different reading conditions. A slide supports a speaker; a manuscript must stand alone. A tutorial asks the reader to act; a report may prioritise a conclusion.

The reusable part is the structured content and analysis. The narrative should still be adapted to its audience.

5.2 What the outputs actually look like

The completed workshop project renders one small analysis to several products.

5.2.1 HTML manuscript

Results

► Code

Table 1: Change scores by study group.

group	participants	mean_change	sd_change
comparison	20	3.0	1.2
intervention	20	10.9	2.7

[Table 1](#) shows a larger average change in the intervention group than in the comparison group.

► Code

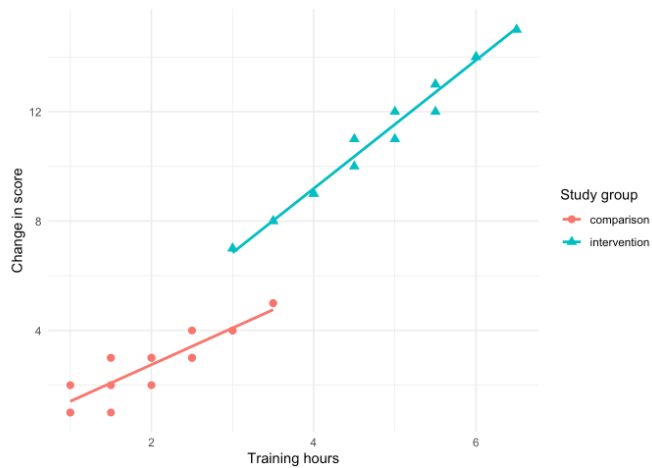


Figure 5.1: HTML manuscript showing a navigable Results section, folded code, a generated table, and a generated figure.

HTML provides navigation, links, responsive layout, and optional folded code.

5.2.2 Standard PDF manuscript

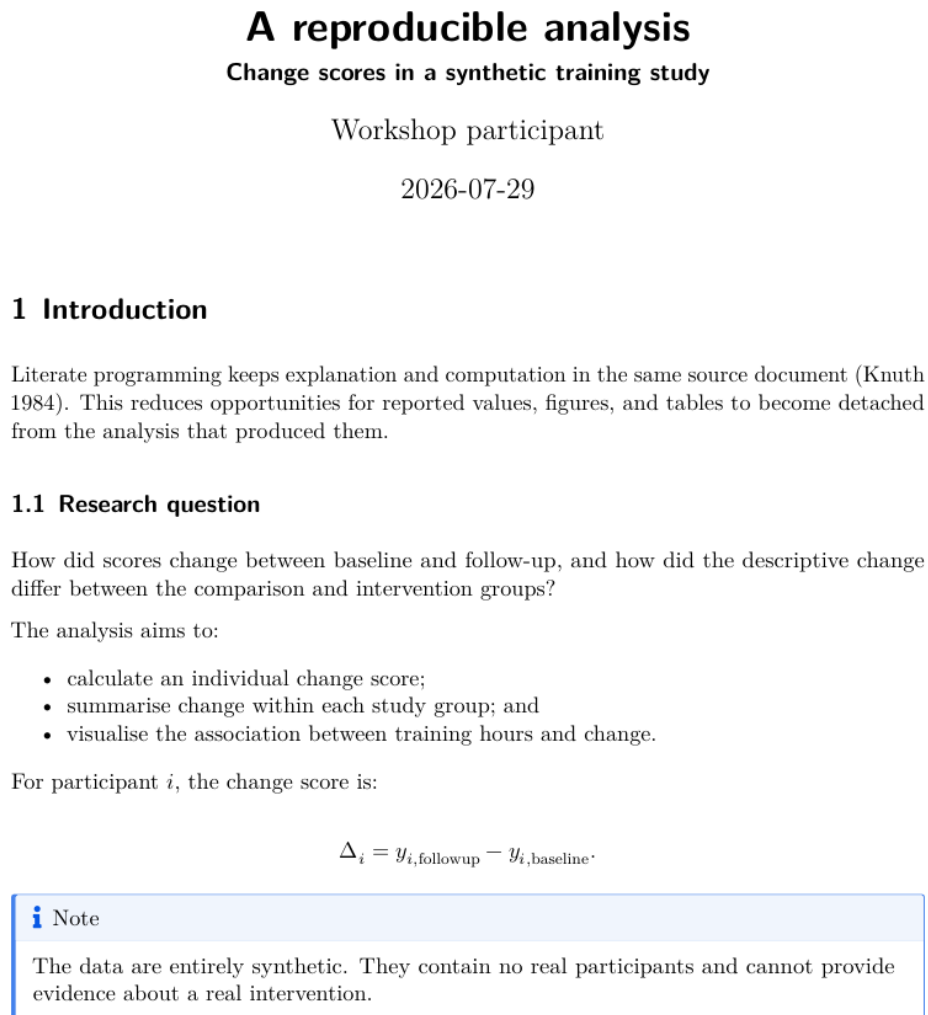


Figure 5.2: First page of the standard PDF manuscript with a conventional title block and fixed page layout.

PDF fixes typography and page boundaries for distribution and review.

5.2.3 Elsevier-format PDF

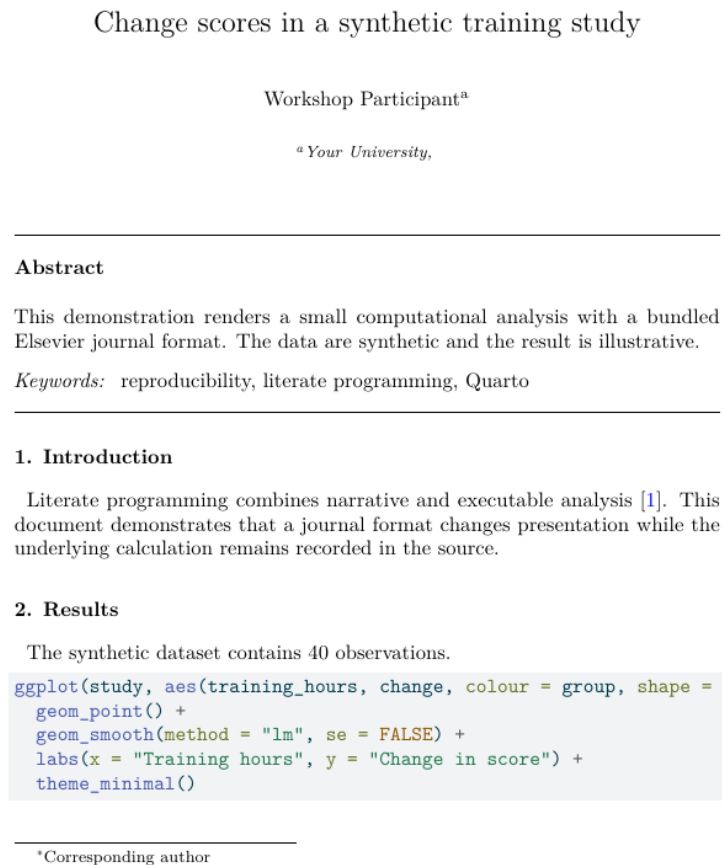


Figure 5.3: First page of the same demonstration analysis rendered with the bundled Elsevier journal format.

The journal format changes title metadata, typography, citations, and layout.

5.2.4 RevealJS presentation

A computed result

The slides read the same 40 observations used by the manuscript.

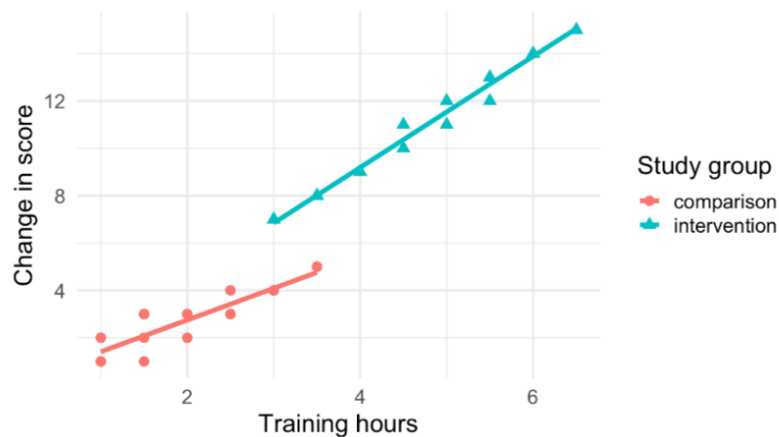


Figure 1



3 / 5

Figure 5.4: RevealJS slide showing the same computed sample size and figure in presentation layout.

The result is reused, but the prose is shortened and the visual is enlarged for a live audience.

5.3 One source with common and format-specific options

The manuscript declares both HTML and PDF:

```
format:
  html:
    toc: true
    code-fold: true
    number-sections: true
  pdf:
    number-sections: true
    keep-tex: true
```

Options indented under `html` affect only HTML. Options indented under `pdf` affect only PDF. Shared document metadata sits above `format`:

```
title: "Change scores in a synthetic training study"
author: "A. Researcher"
bibliography: bibliography.bib
number-sections: true
format:
  html:
    toc: true
  pdf: default
```

Move an option to the shared level only when it is valid and intended for every format.

Render one target:

```
quarto render manuscript.qmd --to html
quarto render manuscript.qmd --to pdf
```

Render all declared targets:

```
quarto render manuscript.qmd
```

5.4 Format is not content

Ideally these remain identical across outputs:

- data transformations;
- model specifications;
- computed values;
- substantive claims;
- figure and table labels;
- citations; and
- section identifiers.

These may legitimately differ:

- navigation and table of contents;
- page breaks;
- whether code can be folded;
- figure dimensions;
- link presentation;
- supplementary navigation;
- publisher metadata; and
- typography.

If changing from HTML to PDF changes a numerical answer, the project probably has an execution, state, or conditional-content problem—not a formatting issue.

5.5 HTML, PDF, and Word serve different workflows

Feature	HTML	PDF	DOCX
Responsive layout	Yes	No	Editor-dependent
Fixed pages	No	Yes	Yes, but may reflow
Foldable code	Yes	No	No
Live hyperlinks	Strong	Supported	Supported
Reader annotation	Browser tools	Strong in review tools	Strong in editorial workflows
Precise typesetting	Theme/CSS	LaTeX or Typst	Reference document/styles
Common use	Web reading	Review and archival copy	Co-author/editor revision

Word output can be useful even when PDF is the final target. A journal or co-author may require tracked changes in DOCX. Quarto can use a reference DOCX to map semantic elements to organisational styles.

Do not promise that every advanced HTML widget or LaTeX command will transfer perfectly to Word. Prefer portable Markdown, figures, tables, equations, and cross-references when multi-format output matters.

5.6 Numbering and cross-references survive reformatting

The section labels introduced in Section 3.4 are format-independent:

```
## Methods {#sec-methods}

See @sec-methods for the study design.
```

Figures and tables use the same model:

```
#| label: fig-primary-result
#| fig-cap: "Primary outcome by study group."
```

```
The primary comparison is shown in @fig-primary-result.
```

The format decides whether the result is “Figure 1”, “Fig. 1”, or another publisher convention. The source refers to a stable identifier rather than a typed number.

5.7 Citation processing

Research publishing involves three separable components:

1. **bibliographic data** in a `.bib` or supported bibliography file;
2. **citation locations** such as `[@knuth1984]` in the source; and
3. **citation style**, supplied by the format or a `.csl` file.

Changing style should not require retyping citations.

```
bibliography: bibliography.bib
csl: journal-style.csl
```

Journal extensions may supply their own CSL, BibTeX style, or citation method. Check the rendered bibliography for:

- author order and spelling;
- article title and journal;
- year, volume, issue, and pages;
- DOI or URL;
- duplicated records; and
- uncited or missing sources.

Automated formatting faithfully reproduces incorrect metadata.

5.8 Author and affiliation metadata

Quarto supports structured authors:

```
author:
- name: Alex Researcher
  orcid: 0000-0000-0000-0000
  email: alex@example.edu
  affiliations:
    - ref: university
affiliations:
- id: university
  name: Example University
  department: School of Research
```

Structured metadata lets a format decide how names, affiliations, email addresses, and ORCID identifiers appear. A hand-typed title-page paragraph is harder for templates and indexing systems to interpret.

5.9 What a journal extension contains

A Quarto journal extension can package:

- a LaTeX document class;
- Pandoc template partials;
- CSL or BibTeX styles;
- publisher-specific YAML defaults;
- Lua filters;
- CSS for an HTML preview; and
- a starter article template.

It changes how Quarto translates structured source into a publisher's format. It should not contain the scientific analysis itself.

5.10 Starting a journal article versus adding a format

For a new article based on a supplied template:

```
quarto use template quarto-journals/elsevier
```

This creates a project with example metadata and source.

For an existing article:

```
quarto add quarto-journals/elsevier
```

This installs the extension under `_extensions`. The workshop bundles that folder so the demonstration works without downloading it on the day.

The document can then declare:

```
format:
  elsevier-pdf:
    keep-tex: true
```

The `keep-tex` option is useful when a submission requires generated LaTeX source or when a template error needs diagnosis.

5.11 Comparing standard and journal output

The standard PDF and Elsevier PDF above contain the same analysis concept but look different because they serve different conventions.

When comparing formats, check:

- title, author, and affiliation layout;
- abstract and keyword placement;
- heading levels and numbering;
- figure size and placement;
- table width;
- citation and bibliography style;
- line and page breaks;
- supplementary-material requirements; and
- files required for submission.

 A template is not the journal instructions

Extensions can lag behind publisher changes, and individual journals can impose requirements beyond a publisher-wide class. Check the journal's current author instructions, accepted source formats, required declarations, word limits, reference style, and figure specifications at the time of submission.

5.12 Conditional content

Occasionally one format needs an explanation or resource that another does not. Quarto supports format-aware content:

```
::: {.content-visible when-format="html"}
Use the navigation panel to explore the analysis.
:::

::: {.content-visible when-format="pdf"}
The full analysis begins on the next page.
:::
```

Use conditional content sparingly. Duplicated claims can drift apart. Prefer shared source unless the reading context genuinely differs.

5.13 Page and figure problems are diagnostic

A plot that works in HTML may be too wide for PDF. A long table may overflow a page. Good repairs include:

- choosing sensible figure dimensions;

- simplifying a figure rather than shrinking all text;
- splitting a table by meaning;
- moving detailed results to an appendix;
- using landscape pages only when necessary; and
- checking the output at its actual reading size.

Avoid format-specific manual spacing and raw LaTeX until semantic options have been exhausted. Raw fixes often make another output worse.

5.14 Reproducible submission workflow

A defensible workflow is:

1. Render standard HTML frequently during analysis.
2. Render PDF regularly rather than only at submission time.
3. Run a clean render after restarting R.
4. Inspect every numbered reference and citation.
5. Render the journal template from the same source.
6. Compare computed values across outputs.
7. Read the journal's current instructions.
8. Package required source, bibliography, class, figure, and supplementary files.
9. Preserve the submitted source and environment record.

5.15 Publishing exercise

Using `manuscript.qmd` and `journal-demo.qmd`:

1. Render the manuscript to HTML and PDF.
2. Compare headings, navigation, code display, figures, tables, and references.
3. Turn section numbering off in HTML only and confirm that cross-references still resolve.
4. Turn numbering back on.
5. Render the Elsevier demonstration.
6. Identify at least five presentation changes and confirm that the sample size and figure data are unchanged.
7. Find which files in `_extensions/quarto-journals/elsevier` control the class, bibliography, and title layout.
8. Write a short submission checklist for a journal relevant to your discipline.

5.15.1 Checkpoint

You can distinguish analysis from format, explain when to start from a template versus add an extension, and verify that multiple outputs remain substantively consistent.

6 Bonus: prove it reruns

The final test is not whether the current HTML looks correct. It is whether the project can recreate it from its sources.

6.1 Clean-session check

1. Save every source file.
2. Restart R.
3. Do **not** run cells interactively.
4. In the project terminal, run:

```
quarto render manuscript.qmd
```

5. Open the newly rendered outputs.

6.2 Reproducibility checklist

Use this checklist before sharing a computational document:

- ☐ Source data are present and documented.
- ☐ All paths are relative to the project.
- ☐ Every reported value is calculated or clearly sourced.
- ☐ Figures and tables are generated by code.
- ☐ Labels and cross-references resolve.
- ☐ Citations and references render.
- ☐ A fresh R session can render the document.
- ☐ Required software and packages are listed.
- ☐ The output contains no private paths, credentials, or unintended data.
- ☐ Figure descriptions and document structure are accessible.

6.2.1 Final challenge 6 minutes

Without running any R cell manually, rebuild the manuscript. Explain to a neighbour how the sample-size sentence stays synchronised with the CSV.

6.2.2 Workshop outcome

You leave with a functioning literate-programming project containing a manuscript. The project reruns from source and can be used as a reference for your next real document.

6.3 Record the environment

The completed manuscript ends with an executable R cell containing:

```
#| label: session-information  
sessionInfo()
```

This record is useful diagnostic evidence. It is not a dependency lockfile, but it tells a future reader which R and package versions produced the output.

7 Next steps

7.1 Apply the workflow to real work

Choose one modest document rather than migrating an entire research programme. A useful first project has:

- one or two input files;
- a calculation you currently repeat;
- at least one figure or table; and
- a clear reader and output format.

Rebuild it as a project with source data, a `.qmd` file, a bibliography, and a short setup note.

7.2 Add safeguards when they solve a problem

Possible next layers include:

- Git for change history and collaboration;
- `renv` for an R package lockfile;
- automated rendering in continuous integration;
- parameterised reports;
- Quarto manuscripts and journal extensions;
- project profiles for student and solution variants;
- a shared institutional theme; and
- publishing to an internal server or static host.

The [Beyond the workshop](#) appendix gives a starting point without making these tools prerequisites.

7.3 Keep these habits

1. Treat rendered documents as products, not the primary source.
2. Keep inputs immutable and transformations explicit.
3. Prefer relative paths.
4. Render from a clean session regularly.
5. Separate computation from claims: code updates numbers, authors remain responsible for interpretation.
6. Preserve enough environment information for another person to diagnose a failed rerun.

7.4 Resources

- [Quarto guide](#)
- [Quarto computations](#)
- [Quarto journal articles](#)
- [Quarto presentations](#)
- [unilur extension](#)

Tip

Copy the completed project, rename the copy, replace the synthetic CSV with your own documented data, and change one section at a time. Keep the original as a known-working reference.

A Markdown and cell-option reference

A.1 Markdown

Purpose	Source
Heading	## Results
Emphasis	*emphasis*
Strong emphasis	**important**
Inline code	<code>`manuscript.qmd`</code>
Link	[label] (https://example.org)
Image	![Alt text] (image.png)
Citation	[@citation-key]
Inline equation	$\$y = a + bx\$$
Footnote	Statement.[¹]
Figure reference	@fig-change
Table reference	@tbl-summary

Lists need a blank line before them:

The aims are:

- estimate the average change;
- compare groups; and
- communicate uncertainty.

A.2 Code cells

An executable R cell begins with ````{r}` and contains:

```
#| label: descriptive-name
#| echo: true
#| warning: false
#| message: false

# R code
```

Option	Common value	Effect
label	fig-change	Identifies the cell/output
echo	false	Hides source code
eval	false	Displays but does not run code
include	false	Runs but hides code and output
output	false	Hides output
warning	false	Hides R warnings
message	false	Hides R messages
fig-cap	text	Adds a caption
fig-alt	text	Adds an accessible description
tbl-cap	text	Adds a table caption

Labels must be unique. Use the `fig-` and `tbl-` prefixes for referenceable figures and tables.

A.3 YAML

```

---
title: "Document title"
author: "Name"
date: today
bibliography: bibliography.bib
format:
  html:
    toc: true
  pdf:
    number-sections: true
execute:
  warning: false
  message: false
---
```

If YAML parsing fails, check indentation, tabs, unmatched quotes, and whether both `---` delimiter lines are present.

A.4 Terminal commands

```

quarto preview
quarto render manuscript.qmd
quarto render manuscript.qmd --to html
quarto render manuscript.qmd --to pdf
```

B Troubleshooting

B.1 Read the first useful error

Rendering errors often cascade. Start near the first line that names your source file, missing package, missing file, or YAML key.

B.2 `quarto: command not found`

- Restart the editor or terminal after installing Quarto.
- Confirm installation with `quarto --version`.
- On managed university computers, Quarto may need to be installed by IT or selected explicitly in the editor's preferences.

B.3 An R package is missing

Run:

```
Rscript setup.R
```

Or install an individual package in R:

```
install.packages("readr")
```

Restart R and render again.

B.4 The CSV cannot be found

- Confirm the terminal is inside the extracted project folder.
- Confirm `data/training-study.csv` exists.
- Use the relative path exactly as written, including letter case.
- Do not use `setwd()` to conceal a project-location problem.

B.5 YAML parsing fails

Check:

- spaces rather than tabs;
- aligned indentation;
- a colon after each key;
- matching quotation marks; and
- opening and closing `---` lines.

Temporarily reduce the YAML to `title` and one `format`, render, then restore one option at a time.

B.6 PDF will not render

1. Confirm HTML works:

```
quarto render manuscript.qmd --to html
```

2. Check for a LaTeX engine:

```
quarto check latex
```

3. If none is found, install TinyTeX in R:

```
install.packages("tinytex")
tinytex::install_tinytex()
```

4. Restart the editor and rerun `Rscript preflight.R`.

If a managed computer blocks installation, continue with HTML and use the provided completed PDF during the format-comparison activity.

B.7 A figure reference shows ?

- Give the producing cell a unique label beginning with `fig-`.
- Add `fig-cap`.
- Refer to the label as `@fig-name`.
- Render the entire document again.

For tables, use a `tbl-` label and `tbl-cap`.

B.8 It worked interactively but not when rendered

The interactive R session contained an object or package that the document did not create or load. Restart R and add the missing setup to the `.qmd` source. This is a useful reproducibility failure, not a reason to avoid clean sessions.

B.9 Recovery route

If you fall behind:

1. save your current file with a different name;
2. copy the relevant `.qmd` from the completed project;
3. render it to rejoin the workshop; and
4. compare files after the session.

C Python and Jupyter translation

The live workshop uses R so the room can share one execution path. Quarto also supports Python through Jupyter.

C.1 Preflight

Check Python and Jupyter:

```
python --version
quarto check jupyter
```

Create an environment using your organisation's preferred tool, then install:

```
python -m pip install jupyter pandas matplotlib
```

Environment creation and dependency locking differ among `venv`, `Conda`, `Poetry`, and `uv`; follow local policy rather than mixing them casually.

C.2 Python cell

In an executable Python cell, write:

```
#| label: load-data-python

import pandas as pd

study = pd.read_csv("data/training-study.csv")
study["change"] = study["followup_score"] - study["baseline_score"]
```

A `.qmd` file containing Python cells uses the Jupyter engine unless another engine is selected explicitly.

C.3 Inline Python

Type the following sentence (the backticks are part of the source):

The study contains ‘`{python} len(study)`’ participants.

C.4 Figure

```
#| label: fig-change-python
#| fig-cap: "Change score by training hours and study group."
#| fig-alt: "Scatterplot of training hours against change score."

import matplotlib.pyplot as plt

for name, group in study.groupby("group"):
    plt.scatter(group["training_hours"], group["change"], label=name)
plt.xlabel("Training hours")
plt.ylabel("Change score")
plt.legend()
```

C.5 Concepts that transfer directly

- Markdown and YAML
- relative paths
- source/output separation
- cell labels and most presentation options
- citations and cross-references
- HTML, PDF, and RevealJS formats
- clean-session rendering

The main differences are environment management, package APIs, and the computational engine.

D Accessibility checklist

Reproducible output should also be usable output.

D.1 Structure

- Use headings in order; do not choose levels for visual size.
- Use real lists and tables rather than spaces or manual line breaks.
- Give links descriptive text rather than “click here”.
- State the document language.

D.2 Figures

Every meaningful figure should have `fig-alt` that describes:

- the chart type;
- the variables and comparison;
- the major visible pattern; and
- information not already supplied by the caption.

Do not begin with “image of”. Decorative images should have empty alt text.

D.3 Colour

- Do not encode groups by colour alone; add shape, labels, or direct annotation where practical.
- Use palettes with adequate contrast.
- Check figures in greyscale.
- Avoid red/green as the only distinction.

D.4 Tables and code

- Keep tables small and give them captions.
- Avoid merged cells where possible.
- Wrap long code lines and avoid screenshots of code.
- Explain what a code example is intended to demonstrate.

D.5 Slides

- Use large text and high contrast.
- Keep one main idea per slide.
- Describe visual material aloud.
- Add speaker notes for required descriptions and facilitation cues.
- Provide the HTML or PDF afterward so participants can revisit the content.

D.6 Final check

Inspect both HTML and PDF. A correct source does not guarantee equivalent accessibility across output formats.

E Beyond the workshop

These tools strengthen the workflow but are intentionally outside the live three-hour session.

E.1 Git

Git records source changes and supports collaboration. Start by tracking source files, data that may be shared legally, the bibliography, and `_extensions`. Ignore generated output unless a publishing workflow requires it.

Never commit credentials, confidential data, or files you do not have permission to distribute.

E.2 `renv`

For an R project:

```
install.packages("renv")
renv::init()
renv::snapshot()
```

This creates a lockfile describing package versions. Collaborators can use `renv::restore()`. Review organisational policy before committing package archives or configuring private repositories.

E.3 Automated rendering

A continuous-integration job can:

1. restore dependencies;
2. run `quarto render`;
3. fail when code or references break; and
4. publish validated static output.

Automation is most useful after the project renders reliably on a clean local session.

E.4 Quarto manuscripts

Quarto manuscript projects connect an article, computational notebooks, and multiple output formats. They are useful when the computational record itself is part of the publication.

E.5 Parameters and profiles

Parameters can create repeated reports for different cohorts or datasets. Profiles can change configuration for student, solution, draft, and production builds. Prefer these mechanisms once repeated manual switches are causing real errors.

Knuth, Donald E. 1984. “Literate Programming.” *The Computer Journal* 27 (2): 97–111. <https://doi.org/10.1093/comjnl/27.2.97>.

Sandve, Geir Kjetil, Anton Nekrutenko, James Taylor, and Eivind Hovig. 2013. “Ten Simple Rules for Reproducible Computational Research.” *PLOS Computational Biology* 9 (10): e1003285. <https://doi.org/10.1371/journal.pcbi.1003285>.